2300321530046 Roll No.: ..... Date 22/08/2025 Page No. 01

Practical Name: WATER JUG PROBLEM Practical No. 01

AIM:- Write a python program to implement Water Jug Problem

THEORY:- ✱ We are given two jugs with their capacities (in litres).
✱ The task is to determine a series of pouring operations to measure a specific target amount of water.

GIVEN:- (i) There is unlimited water available.
(ii) There is no marking on the jugs.
(iii) Both the jugs are empty in the initial stage.

OPERATIONS AVAILABLE:
(i) Empty a jug completely.
(ii) Fill a jug completely.
(iii) Pour water from one jug to another.

ALGORITHM:
Step(i) Initially, both the jugs are empty.
Step(ii) Check if Jug1 is empty, then fill Jug1 with it's maximum capacity.
Step(iii) Check if Jug2 is full, then empty jug2.
Step(iv) Pour the minimum of (Jug2_max_capacity - jug2_current_water, jug1) from Jug1 to Jug2.
Step(v) Repeat step 2-4 until either one of the jug water levels matches the target quantity.

2300321530046 22/08/2025
Roll No.: ..... Date..... Page No. 02

Practical Name: WATER JUG PROBLEM Practical No. 01

PRODUCTION RULES:

State Space :- $x = 4$, $y = 3$

Rule (i) $(x, y) \rightarrow (4, y)$
(ii) $(x, y) \rightarrow (x, 3)$
(iii) $(x, y) \rightarrow (x-4, y)$, $x > 0$
(iv) $(x, y) \rightarrow (x, y-d)$, $y > 0$
(v) $(x, y) \rightarrow (0, y)$
(vi) $(x, y) \rightarrow (x, 0)$
(vii) $(x, y) \rightarrow (4, y-(4-x))$, $y > 0, x+y >= 4$
(viii) $(x, y) \rightarrow (x-(3-y), 3)$, $x > 0, x+y >= 3$
(ix) $(x, y) \rightarrow (x+y, 0)$ $x+y <= 4$
(x) $(x, y) \rightarrow (0, x+y)$ $x+y <= 3$.

x	y	Rule
0	0	-
0	3	(ii)
3	0	(ix)
3	3	(ii)
4	2	(vii)
0	2	(v)
2	0	(ix)

← Implementation

PYTHON CODE:

```
def gcd(a,b):
    while b:
        a, b = b, a%b
    return a
def display_state(jug1, jug2):
    print(f"{'JUG1: ':>12}{jug1:<6}|{'JUG2: '
        :>12}{jug2}")
```

Roll No.: 2300321530046 Date: 22/08/2025 Page No. 03

Practical Name: WATER JUG PROBLEM Practical No. 01

```
def pour_water(jug1, jug2, capacity1, capacity2):
    pour_amount = min(jug1, capacity2 - jug2)
    jug1 -= pour_amount
    jug2 += pour_amount
    return jug1, jug2

def water_jug_solution(capacity1,capacity2,target_amount):
    jug1=0
    jug2=0
    print(f"Jug 1 Capacity:{capacity1} | Jug 2 Capacity:
        {capacity2}")

    while jug1 != target_amount and jug2!=target_amount:
        if jug1==0:
            jug1 = capacity1
        elif jug2 == capacity2:
            jug2 = 0.
        else:
            jug1, jug2 = pour_water(jug1,jug2,capacity1,
                capacity2)
        display_state(jug1, jug2)

first_jug = int(input("Enter the capacity of the first jug:"))
second_jug = int(input("Enter the capacity of the seond jug:"))
target_amount = int(input("Enter the target litres of water:"))

if target_amount < first_jug or target_amount<second_jug:
    if target_amount % gcd
        (first_jug, second_jug) == 0:
        water_jug_solution(first_jug, second_jug,
            target_amount)
```

```
    else:
        print("This is not possible...")
else:
    print("This is not possible...")
```

OUTPUT:

```
Enter the capacity of the first jug: 4
Enter the capacity of the second jug: 3
Enter the target litres of water: 2
Jug 1 Capacity: 4 | Jug 2 Capacity: 3
    JUG 1: 4      |     JUG 2: 0
    JUG 1: 1      |     JUG 2: 3
    JUG 1: 1      |     JUG 2: 0
    JUG 1: 0      |     JUG 2: 1
    JUG 1: 4      |     JUG 2: 1
    JUG 1: 2      |     JUG 2: 3
```

2300321530046 29/08/2025
Roll No.: ..... Date..... Page No. 05

Practical Name: BREADTH FIRST SEARCH Practical No. 02

AIM:- Write a python program for breadth first search.

CODE:-

```
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
# Example usage
if __name__ == "__main__":
    graph = {
        'A': ['B', 'C'],
        'B': ['A', 'D', 'E'],
        'C': ['A', 'F'],
        'D': ['B'],
        'E': ['B', 'F'],
        'F': ['C', 'E']
    }
    print("Breadth First Search starting from vertex A:")
    bfs(graph, 'A')
```

OUTPUT:

```
Breadth First Search starting from vertex A:
A B C D E F
```